arXiv:2006.09265v1 [cs.LO] 13 Jun 2020

Modelling High-Level Mathematical Reasoning in Mechanised Declarative Proofs

Wenda Li
University of Cambridge
wl302@cam.ac.uk

Lei Yu
DeepMind
leiyu@google.com

Yuhuai Wu
University of Toronto
ywu@cs.toronto.edu

Lawrence C. Paulson
University of Cambridge
lp15@cam.ac.uk

Abstract

Mathematical proofs can be mechanised using proof assistants to eliminate gaps and errors. However, mechanisation still requires intensive labour. To promote automation, it is essential to capture high-level human mathematical reasoning, which we address as the problem of generating suitable propositions. We build a non-synthetic dataset from the largest repository of mechanised proofs and propose a task on causal reasoning, where a model is required to fill in a missing intermediate proposition given a causal context. Our experiments (using various neural sequence-to-sequence models) reveal that while the task is challenging, neural models can indeed capture non-trivial mathematical reasoning. We further propose a hierarchical transformer model that outperforms the transformer baseline.

1 Introduction

Mathematical proof can mostly be considered as a sequence of intermediate propositions. Consider the following prose proof:

Proof of irrationality of $\sqrt{2}$. Assume $\sqrt{2}$ is rational. Then there exists a pair of coprime integers a and b such that $\sqrt{2} = a/b$, and it follows that $2 = a^2/b^2$ and then $2b^2 = a^2$. Hence a^2 is even, and therefore a is even. Thus there exists c such that $a = 2c$, which combined with $2b^2 = a^2$ yields $2c^2 = b^2$: hence b is also even. So a and b are both even although they are coprime, contradiction. □

By intermediate propositions, we refer to each of the reasoning steps, such as '$\sqrt{2}$ is rational', '$2 = a^2/b^2$', that are connected causally by words such as 'therefore', 'since', 'yields'. The implicit gaps between these intermediate propositions are neglected in human reasoning as they are often too primitive and obvious to go through explicitly. However, those implicit derivations sometimes contain fatal errors. As a result, proof assistants, such as Coq [5] and Isabelle [38], were developed to mechanically check proofs down to primitive inferences or axioms. Over the last decade, non-trivial mathematical theorems [16, 18, 19], security protocols [14, 40], and industry-scale software systems [27, 32] have been mechanically checked, highlighting numerous issues and bugs. Despite those successes, mechanisation still requires substantial human effort: in the seL4 project [27], 20 person-years were devoted to verify an OS kernel written in 8700 lines of C. To boost the automation in proof assistants, approaches have been proposed ranging from incorporating automated theorem provers [6–8] to synthesising tactics [3, 37, 43, 53] using learning-based methods.

However, existing learning-based automation attempts [3, 53] are mostly machine-oriented: numerous primitive inferences or hard-coded tactics are explored to justify a human-provided conjecture and

Preprint. Under review.

```
1  theorem "sqrt 2 ∉ ℚ"
2  proof
3    assume "sqrt 2 ∈ ℚ"
4    then obtain a b :: int where
5        "sqrt 2 = a/b" "coprime a b" sledgehammer
6    then have "2 = a² / b²" sledgehammer
7    then have *:"2*b² = a²"
8    proof -
9      assume "2 = a² / b²"
10     then have "b² * (2::real) = a²" sledgehammer
11     then show "2 * b² = a²" sledgehammer
12   qed
13   then have "even a" sledgehammer
14   then obtain c::int where "a=2*c" sledgehammer
15   with * have "b² = 2*c²" sledgehammer
16   then have "even b" sledgehammer
17   with ‹even a› ‹coprime a b› show False sledgehammer
18 qed
```

```
theorem "sqrt 2 ∉ ℚ"
  apply clarsimp
  apply (elim Rats_abs_nat_div_natE)
  apply (drule arg_cong[where f="λx. x * x"])
  apply (simp add:nonzero_eq_divide_eq)
  apply (subgoal_tac "real (2*(n*n)) = real (m*m)")
  apply (subst (asm) of_nat_eq_iff)
  apply simp_all
  apply (simp flip:power2_eq_square)
  apply (subgoal_tac "even m")
  apply (elim evenE)
  apply (subgoal_tac "even n")
  apply simp_all
  apply (metis dvd_triv_left even_power)
  apply (metis dvd_refl even_mult_iff power2_eq_square)
  done
```

Figure 1: Full declarative proof (left) and tactic proof (right) of the irrationality of $\sqrt{2}$ in Isabelle/HOL.

produce a machine-checkable but illegible proof. Those proofs are often called *tactic proofs*. In contrast, we approach automation in a human-oriented way: synthesising high-level intermediate propositions in a proof style called *declarative proof* style. A comparison of two proof styles is shown in Fig. 1, both proving the irrationality of $\sqrt{2}$. One can see that the declarative proof (left) resembles the human prose proof with clear high-level structures, whereas the tactic proof (right) is difficult to comprehend without domain knowledge of the tactics. We argue that a human-oriented approach is an essential step to further automation. When searching for a proof, machine-level exploration can easily get lost due to the astronomical number of primitive steps, while high-level jumpy reasoning largely reduces the effective search space. As will be illustrated in §8, existing tactic-synthesis frameworks are limited by the inability to generate intermediate propositions. The high-level reasoning approach is also complementary to existing tactic based works.

In this work, we have mined arguably the largest publicly-hosted repository of mechanised (declarative style) proofs: the Achieve of Formal Proofs (AFP).[1] The AFP is checked by the Isabelle proof assistant [38] and contains 143K lemmas (increasing by about 10K annually) contributed by 347 authors. Combining the AFP with the Isabelle/HOL library yields a dataset of 204K formally-proved lemmas, which is much larger than (the number of lemmas used in) the previous tactic-oriented benchmarks (71K in CoqGym [53], 11K in HolStep [25], 29K in HOList [3], 1.6K in GamePad [22]).[2] The dataset covers a broad spectrum of subjects, including foundational logic (e.g. Gödel's incompleteness theorems [39]), advanced analysis (the Prime Number Theorem [11]), computer algebra [45], cryptographic frameworks [26, 33], and data structures [10, 29, 36].

The mined dataset allows us to propose a proposition synthesis task — IsarStep.[3] In this task (§3), a model is asked to propose meaningful steps in unseen proofs by reading derivations in existing proofs. For example, given $\sqrt{2} = a/b$ and $2b^2 = a^2$, we would like to synthesise a meaningful intermediate proposition $2 = a^2/b^2$. To succeed in this task, the model is required to learn the meaning of important mathematical concepts (e.g. determinant in linear algebra, residue in complex analysis), how they are related to each other through theorems, and how they are utilised in proofs. In addition to the contribution to the theorem proving community on improving proof automation, we believe that our proposed task will also contribute to the machine learning community by providing a benchmark for testing and advancing machine learning algorithms on mathematical reasoning.

We frame the proposed task as a sequence-to-sequence (seq2seq) prediction problem. Beyond evaluating the existing neural seq2seq model baselines: the seq2seq with attention [2], the transformer [46], we also propose a new architecture: hierarchical transformer[4] (§5). The architecture is motivated by the way humans reason about propositions; it consists of a set of local transformer layers, modelling the representation of each proposition, and a set of global layers, modelling the correlation across propositions. Experiments (§6) reveal that these neural models can solve 10–20% of problems on the test set on average, in which the hierarchical transformer achieves the best result. Further analysis (§7) on the output of these models shows that while the proposition synthesis task is hard, the neural models can indeed capture mathematical reasoning. We find that the embeddings of closely related

[1] `https://www.isa-afp.org`

[2] Tactic-oriented tasks are usually more primitive so each lemma can be split into more data points.

[3] The dataset will be publicly available at xxxx.

[4] The code will be released at xxxx.

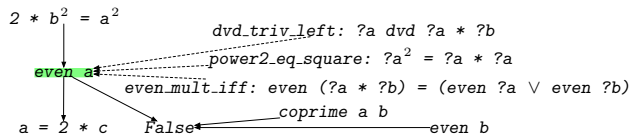

Figure 2: Causal environment around `even a`, where a filled arrow and a dotted arrow, respectively, refer to implications from a (local) intermediate proposition and a (global) named lemma.

mathematical concepts are close in cosine space; models can reason about the relation between set, subset, and member, and more complicated multi-step reasoning that is even difficult for humans to follow.

2 Technical Background

Most proof assistants support *tactics* (e.g., `clarsimp`, `elim` and `drule` in Fig. 1, right) that transform *proof states*. A conjecture is considered mechanically proved if a proof state starting with the conjecture itself has been transformed, by a sequence of tactics, into a state with no subgoals left. Therefore, as has been shown, a tactic proof is a sequence of tactics optionally with some arguments (see Appendix A for additional details).

Declarative proofs emulate prose proofs by focusing on intermediate propositions. They differ from the prose ones in that all claims require explicit justifications. These justifications can be found automatically by *Sledgehammer* [7], which internally invokes automatic theorem provers [4, 9, 41]. When automatic justifications fail, users can open another proof block (e.g., line 8-12 in Fig. 1) and provide extra intermediate propositions.

3 The IsarStep Task

In order to imitate human mathematical reasoning, we focus on the synthesis of intermediate propositions in declarative proofs using machine learning, a different approach from prior works on tactic synthesis [3, 37, 43, 53]. We mined a dataset of mechanised declarative proofs from the AFP and the Isabelle/HOL [38] standard library and defined a proposition generation task: the IsarStep.

Key to high-level mathematical reasoning is to bridge the gap in causal relations using intermediate propositions. For example, in the $\sqrt{2}$ proof, the proposition `even a` is conjectured to fill in the gap between `2 * b² = a²`, `a = 2 * c` and `False` (see Fig. 2, which was extracted from highlighted parts in Fig. 1, left). Deriving `False` also requires two other local propositions, `coprime a b` and `even b`, and justifying `even a` requires three library lemmas (`dvd_triv_left`, `power2_eq_square` and `even_mult_iff`), which were found by Sledgehammer. In this case, `even a` is the target proposition while other surrounding propositions and lemmas form the context.

In our defined IsarStep task, each example is formed by five parts:

- **F.1** a target proposition (e.g. `even a`),
- **F.2** a set of used local propositions to derive **F.1** (e.g. `2 * b² = a²`),
- **F.3** a set of local propositions derived from the target proposition **F.1** (`a = 2 * c` and `False`),
- **F.4** other local propositions and (global) lemmas used to justify **F.3** (`even b` and `coprime a b`),
- **F.5** a set of used (global) lemmas to justify **F.1** (e.g. `dvd_triv_left`).

We want to synthesise **F.1** given **F.2** – **F.4** with **F.5** optional: the named lemmas in **F.5** are common knowledge and can be used as additional hints. The propositions are generated as a sequence of tokens and therefore the search space is Σ^*: search over 28K actions (§4.3, vocabulary size for seq2seq models) at every timestep without a predefined maximum output length. IsarStep can be considered as single step reasoning, which can be repeated to sketch more complex proofs. This task provides a vehicle for machine learning models to imitate causal reasoning in mathematics.

4 Dataset Preprocesssing and Statistics

The mined raw dataset has long propositions and a large number of unique tokens. To alleviate the performance deterioration of machine learning models due to the aforementioned problems, we propose tricks to preprocess the raw dataset, including free variable normalisation and removing unnecessary parentheses, which substantially reduce the sequence lengths and vocabulary size.

4.1 The Logic and Tokens

The core logic of Isabelle/HOL is simply-typed λ-calculus with de Bruijn indices for bound variables [49, Chapter 2.2]. A local proposition or a (global) lemma/theorem is essentially a *term* in the calculus. As types can be inferred automatically, we drop types in terms (to reduce the size of the vocabulary) and encode a term as a sequence of tokens that include lambda term constructors: `CONST`, `FREE`, `VAR`, `BOUND`, `ABS` (function abstraction), and `$` (function application). Additionally, parentheses have been used in the sequence to represent the tree structure. To give an example, we encode the proposition *`even a`* as the following sequence of tokens separated by a white space:

```
CONST HOL.Trueprop $ ( CONST Parity.semiring_parity_class.even $ FREE <X0> )
```

where `CONST HOL.Trueprop` is a boilerplate function that converts from type *`bool`* to *`prop`*; `CONST Parity.semiring_parity_class.even` is the *`even`* predicate; `FREE <X0>` encodes the Skolem constant `a` in *`even a`*. Since `a` is a user-introduced local constant that can be arbitrary, we normalised it to the algorithmically generated name `<X0>` in order to reduce the vocabulary size (see §4.2).

Overall, every local proposition and global lemma/theorem is encoded as a sequence of tokens, and can be mostly decoded to the original term with type inference.

4.2 Free Variable Normalisation

Due to Isabelle's use of de Bruijn indices, bound variables have already been normalised: $\forall$ *`x. P x`* is no different from $\forall$ *`y. P y`*, as both *`x`* and *`y`* are encoded as `BOUND 0`. However, arbitrary variable names can be introduced by the command **fix** in declarative proofs or unbounded variables in lemma statements (e.g. *`False ⟹ P`* and *`False ⟹ Q`* are semantically equivalent but with different free variables). To reduce the vocabulary size here, we normalised these free variables like the bound ones. For example, *`False ⟹ P`* would be normalised to *`False ⟹ <V0>`* as *`P`* is the first free variable in the proposition. Such normalisation reduced the vocabulary size by *one third*. The normalisation preserves the semantics, and we can always parse a normalised term back under a proper context.

4.3 Statistics

We have mined a total of 850K data points for IsarStep. We removed examples in which the length of the concatenation of the source propositions, i.e. **F.2** – **F.4** in §3, longer than 800 and the length of the target propositions, i.e. **F.1** in §3, longer than 200, which results in approximately 665K examples. From these examples we randomly sampled 8000 examples for validation and 8000 examples for testing. We removed duplicates and those whose target propositions exist in the training data. The final dataset split is 609K, 4143, 4145 for the training, validation, and test sets, respectively. The vocabulary size is 28,370.

5 Model

We define $\underline{\boldsymbol{x}} = [\boldsymbol{x}^1, \boldsymbol{x}^2, \ldots, \boldsymbol{x}^I]$ as the sequence of source propositions with I propositions, and $\boldsymbol{y} = (y_1, y_2, \ldots, y_N)$ as the target proposition containing N tokens. Let $\boldsymbol{x}^i = (x_1^i, x_2^i, \ldots, x_M^i)$ represent the ith proposition in the set, consisting of M tokens. Each source proposition $\boldsymbol{x}^i$ belongs to a category **F.2** – **F.4** defined in §3. We annotate the category corresponding to $\boldsymbol{x}^i$ as $\mathcal{C}_i$ and therefore the sequence of categories corresponding to $\underline{\boldsymbol{x}}$ is $\boldsymbol{\mathcal{C}} = [\mathcal{C}_1, \mathcal{C}_2, \ldots, \mathcal{C}_I]$. The generation of a target proposition $\boldsymbol{y}$ is determined by finding the proposition $\hat{\boldsymbol{y}}$, where $p(\hat{\boldsymbol{y}} \mid \underline{\boldsymbol{x}}, \boldsymbol{\mathcal{C}})$ is optimal,

$$\hat{\boldsymbol{y}} = \arg\max_{\boldsymbol{y}} p(\boldsymbol{y} \mid \underline{\boldsymbol{x}}, \boldsymbol{\mathcal{C}}). \tag{1}$$

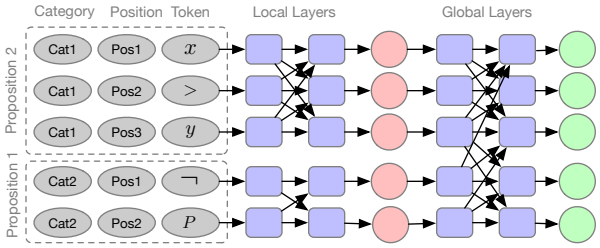

Figure 3: Architecture of the encoder of the hierarchical transformer (HAT). There are two types of layers, the local layers model the correlation between tokens within a proposition, and the global layers model the correlation between propositions. The input to the network is the sum of the token embedding, the positional information, and the embedding of the corresponding category.

We propose two approaches to parameterising the conditional probability $p(\boldsymbol{y} \mid \underline{\boldsymbol{x}}, \boldsymbol{\mathcal{C}})$, which differ in the way of modelling the sequence of source propositions. The first method is simply appending a label to each source proposition indicating their category and then concatenating the source propositions using a special token `<SEP>`, treating the resulting long sequence as the input to a seq2seq model.

Our second approach models the encoding of source propositions hierarchically. As shown in Fig. 3, the encoder has two types of layers. The local layers build the proposition representations by modelling the correlations of tokens within each proposition; the global layers take the proposition representations as input and model the correlations across propositions. Both the local layers and global layers are transformer layers [46]. Positional information is encoded separately for different source propositions. That is, suppose $\boldsymbol{x}^1$ has M tokens, then the position of the first token in $\boldsymbol{x}^2$ is not $M+1$ but 1. The embedding of a token x_m^i is obtained by adding the token embedding, the positional information, and the embedding of the category that the proposition $\boldsymbol{x}^i$ belongs to. The category embedding is learnt together with the rest of the network. We call this model the *hierarchical transformer* (HAT). Intuitively, HAT models the structure of the source propositions more explicitly compared to the first approach and therefore should be better at capturing mathematical reasoning between source and target propositions. We will validate our hypothesis in §6.

6 Experiments

We benchmark three models on IsarStep (§3), namely the seq2seq model with attention (RNNSearch) [2, 52], transformer [46], and hierarchical transformer (HAT). The input to the RNNSearch and the transformer is a concatenation of source propositions (the first parameterisation approach described in §5). We train these models with the same training data and report their performance on test sets.

6.1 Experimental Setup

For RNNSearch[5] [2, 52], we use 2-layer LSTMs [20] with 512 hidden units and 0.2 dropout rate. The hyperparameters for training the transformer[6] are the same as *transformer base* [46], i.e. 512 hidden size, 2048 filter size, 8 attention heads, and 6 layers for both the encoder and decoder. The hyperparameters for HAT are the same, except that the number of local context layers is 4 and global context layers is 2. We share the source and target token embeddings for all the three models. We use beam search decoding with beam size 5 (for top1 accuracies) and 10 (for top10 accuracies). The configurations for different models are the best ones we found based on validation performance. We train these models for 100K steps and pick the checkpoint with the best BLEU on the validation set to evaluate on the test set. Training the transformer and HAT takes 48 hours on 4 Tesla-V100 GPUs.

[5]Codebase: `https://github.com/tensorflow/nmt`

[6]Codebase: `https://github.com/THUNLP-MT/THUMT/tree/pytorch`

Table 1: Test set accurarcies on the IsarStep task. We report mean accuracies over 3 runs (standard deviations in parentheses).

Model	Top-1 Acc.		Top-10 Acc.	
	Base	**+F.5**	Base	**+F.5**
RNNSearch	10.50 (0.14)	12.33 (0.25)	22.86 (0.61)	26.64 (0.29)
Transformer	17.30 (0.08)	17.63 (0.32)	28.33 (0.54)	28.63 (0.45)
HAT	**20.10** (0.35)	**21.10** (0.33)	**33.03** (0.47)	**34.53** (0.17)

6.2 Results

We consider an output sequence as correct if it matches the target sequence exactly at surface form. The top-1 accuracy is then defined as the percentage of the best output sequences that are correct in the given dataset. The top-10 accuracy is defined as the percentage of target sequences appearing in the top 10 generated sequences. Table 1 presents the results of different models for the IsarStep task. We report mean accuracies over 3 runs (standard deviations in parentheses). Overall, the neural seq2seq models achieve around 10–20% top-1 accuracies and 22–34% top-10 accuracies, which indicates that this task is non-trivial and yet not too difficult for neural networks. Of the three models, the transformer [46] outperforms the RNNSearch [2, 52] significantly and our HAT performs best. As mentioned in §3, adding **F.5** is optional and is conjectured for better performance due to exploiting used lemmas explicitly. We experimented both cases and found adding this extra information indeed leads to further improvement.

To explore how well models perform on examples with various target sequence lengths, we categorise the examples on the IsarStep test set into 5 buckets based on their lengths and calculate the accuracies for different buckets. As shown in Fig. 4, the accuracies of all three models decrease as the target sequence lengths get longer. HAT is superior over the transformer on sequences shorter than 120 but is on par with the transformer for longer ones.

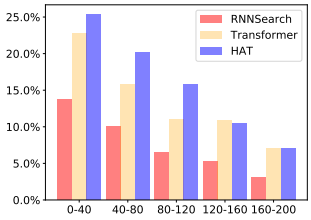

Figure 4: Accuracy of different target sequence lengths.

We subsequently investigate the effect of incorporating the category information for source propositions into the models by removing the category embedding for the input to the HAT encoder (Fig. 3), i.e. we are now modelling $p(\boldsymbol{y} \mid \underline{\boldsymbol{x}})$ instead of $p(\boldsymbol{y} \mid \underline{\boldsymbol{x}}, \boldsymbol{\mathcal{C}})$. We see a dramatic drop in accuracy: 14.6 versus 20.1 obtained by the HAT with category embedding included, indicating the importance of category information. This is in line with human proofs: without causal relations, gap-filling intermediate propositions serve no purpose.

7 Analysis

In this section, we present an in-depth analysis of what has been learnt by the neural network models. To summarise our findings: 1) the seq2seq models can learn the syntax of propositions correctly; 2) the learned token embeddings are comprehensive in that related mathematical concepts are close in cosine space; 3) manual inspection of the generated propositions reveal that models can learn non-trivial mathematical reasoning and even more complicated multi-step reasoning.

Token Embeddings To investigate whether the seq2seq models have learnt mathematical reasoning, we checked whether the learnt token embeddings were meaningful. We first projected the learnt embeddings for all the tokens in the vocabulary into a three-dimensional space via principal component analysis and chose random tokens and checked their 50 nearest neighbours in cosine distance. We found that the embeddings of related concepts in mathematics were close, indicating that the models have managed to learn the relations between mathematical concepts — the basic step towards reasoning mathematically. For example, in Fig. 5, the neighbours of 'Borel measurable' are mostly measure theory related including 'almost everywhere', 'integrable', and 'null set', while 'arrow' is close to 'isomorphism' (EpiMonoIso.category.iso), 'identity'(Category.partial_magma.ide), and 'inverse arrow'(EpiMonoIso.category.inv), which are concepts in category theory. Additionally, vector

arithmetic also seems to connect related mathematical definitions: for example, the three closest tokens next to 'bounded' + 'closed' are 'bounded','closed', and 'compact', where compactness can be alternatively defined as boundedness and closedness (on a Euclidean space).

Attention Visulisations We next investigate how reasoning has been learnt by visualising attentions from the transformer model [46]. We find that important and related tokens are likely to attend to each other. For example, Fig. 6 illustrates the visulisation of the last layer of the transformer encoder for the source propositions **F.2**: **F.3**: $x_{70} \in x_{39}$ **F.4**: $x_{57} \subseteq x_{39}$. The target proposition generated from the model is $x_{70} \in x_{57}$. The interpretation of those source propositions is that combining with (**F.4**) $x_{57} \subseteq x_{39}$ we would like to infer the intermediate step so that the goal $x_{70} \in x_{39}$ can be proved. The transformer model gives the correct answer $x_{70} \in x_{57}$ which implicitly applied the lemma

$$x \in A, A \subseteq B \vdash x \in B \tag{2}$$

that relates $\in$ and $\subseteq$. On the last self-attention layer of the transformer encoder (Fig. 6), $\in$ and $\subseteq$ attend to each other. Interestingly, the above reasoning seems robust. If we swap x_{57} and x_{39} in **F.4** (i.e., the source is now **F.2**: **F.3**: $x_{70} \in x_{39}$ **F.4**: $x_{39} \subseteq x_{57}$), the answer becomes $x_{70} \in x_{39}$. This totally makes sense since (2) no longer applies (despite that $\in$ and $\subseteq$ still attend to each other similarly as in Fig. 6) and $x_{70} \in x_{39}$ can only be discharged by proving itself.

Multi-Step Reasoning By further inspecting the generated propositions, we find that the model can implicitly invoke multiple theorems as humans normally do. While this property can be found in quite a few examples, here we show one of them due to the limited space. We refer the readers to the appendix for more examples. Given the source **F.2**: $\dim(\mathrm{span}(x_0)) \leq \mathrm{card}(x_2)$ **F.3**: $\mathrm{card}(x_2) = \dim(x_0)$ **F.4**: $\mathrm{card}(x_2) \leq \dim(x_0), \mathrm{finite}(x_2)$, where dim, span and card refer to the dimensionality, the span, and the cardinality of a set of vectors, respectively, and the model gives the correct answer $\dim(x_0) \leq \mathrm{card}(x_2)$. Here, $\dim(x_0) \leq \mathrm{card}(x_2)$ is derived by $\dim(\mathrm{span}(x_0)) \leq \mathrm{card}(x_2)$ only if the model has implicitly learned the following theorem $\vdash \dim(\mathrm{span}(S)) = \dim(S)$, while $\dim(x_0) \leq \mathrm{card}(x_2)$ yields $\mathrm{card}(x_2) = \dim(x_0)$ (in conjunction of $\mathrm{card}(x_2) \leq \dim(x_0)$) only if the model has implicitly invoked the antisymmetry lemma $x \leq y, y \leq x \vdash x = y$.

Understandable Mistakes Surprisingly, some of the incorrect answers given by the transformer model still make sense. For example, given the source **F.2**: $\exists x \subseteq x_5(x_8).\, x \neq \{\} \wedge x \subset_{x_0} x_5(x_8)$ **F.3**: $x_9 = \{x_{10}\}$ **F.4**: $x_5(x_8) = \{x_{10}\}$, $x_{10} \in x_5(x_8)$, $x_{10} \in x_9$, where $\subset_{x_0}$ is a special binary relation on finite sets. The target answer is $x_9 \subseteq x_5(x_8)$ where x_9 is a new fixed variable introduced by the existential proposition in **F.2**. The target is a valid answer because it can obviously be derived from **F.2** and combined with propositions in **F.4** to derive **F.3**: putting together $x_9 \subseteq x_5(x_8) = \{x_{10}\}$ and $x_{10} \in x_9$ yields $x_9 = \{x_{10}\}$. The model gives an alternative derivation of **F.2**: $x_9 \subset_{x_0} x_5(x_8)$, which can still be related to **F.3** by a property of $\subset_{x_0}$: $A \subset_R \{x\} \vdash x \notin A$. Applying this property to $x_9 \subset_{x_0} x_5(x_8) = \{x_{10}\}$ yields $x_{10} \notin x_9$, which, in conjunction with $x_{10} \in x_9$, leads to $x_9 = \{x_{10}\}$. In this particular example, the model might just have chosen a less straightforward step to bridge the gap between **F.2** and **F.3**.

Failures Some of the incorrect answers appear related to the difficulty of modelling fixed and unbounded variables (e.g. x_{10} and v_2), whose distributed representation vary from context to context (unlike global constants like 'Borel measurable'). For example, consider the source **F.2**: **F.3**: $\mathrm{poly}(x_1^{x_2}) = \mathrm{poly}(x_3)$ **F.4**:, where the function poly takes a polynomial p and a value a, and returns the evaluation $p(a)$. To derive the equality between two functions in **F.3**, we naturally invoke functional extensionality and try to prove these two functions are equal on all inputs: $\mathrm{poly}(x_1^{x_2}, v_0) = \mathrm{poly}(x_3, v_0)$, as suggested by the reference answer. Here, the reference answer introduces a new variable v_0 that does not appear in the source, which probably causes problems to our model that instead gives a non-sensible answer: $v_0 \in \{\} \rightarrow \exists x.\, x_1(v_0, x)$.

8 Related Work

Our work and the recent abundant work in tactic synthesis on Coq [43, 53], HOL Light [3, 37], and HOL4 [13] are complementary. A key limitation in existing work is that the tactic arguments can only be a previously proved lemma, a small integer (e.g., within $[-10, 10]$) or a sub-term from the proof state; an *arbitrary* proposition/term as an argument is beyond the power of current tactic synthesis

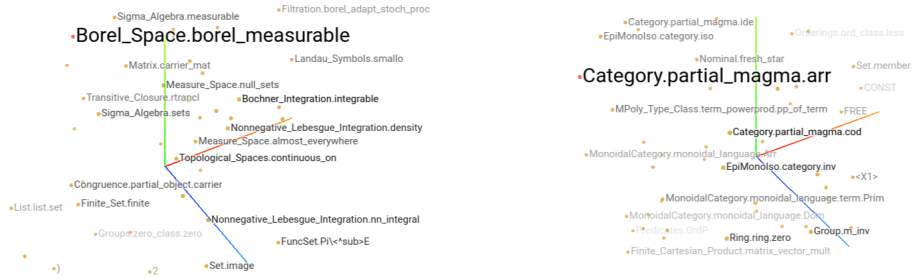

Figure 5: Nearest neighbours of the tokens ‘Borel measurable’ (left) and ‘arrow’ (right) in cosine space. The 512-dimensional embeddings are projected into 3-dimensional embeddings. Neighbours are found by picking the top 50 tokens whose embeddings are closest to the selected token.

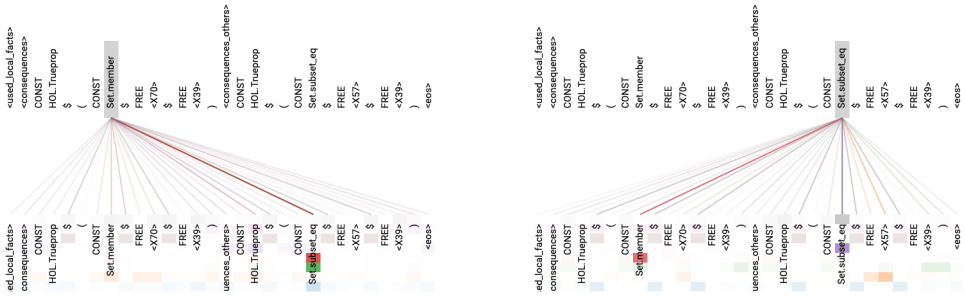

Figure 6: Attention visualisation of the last layer of the transformer encoder for the source propositions **F.2**: **F.3**: $x_{70} \in x_{39}$ **F.4**: $x_{57} \subseteq x_{39}$. The generated target proposition is $x_{70} \in x_{57}$.

frameworks. This limitation prohibits current frameworks from synthesising the tactic-based proof in Fig. 1 due to its introduction of arbitrary propositions (via the `subgoal_tac` tactic). The ability to synthesise intermediate propositions can well complement current tactic synthesis frameworks. In the meantime tactic synthesis can be used to justify proposed intermediate propositions, which complements the *Sledgehammer* command we used in §2. Finally, our dataset is larger than previous ones in terms of the number mechanised lemmas, but it is not yet an interactive environment like CoqGym [53] or HOList [3].

A classic machine learning task for theorem proving is *premise selection*: to select a few lemmas that are likely to be useful for the current proof from a large library of previously proved lemmas. Various methods have been used for this task, ranging from hand-crafted features and distance functions [21, 35], to classic machine learning methods like Naive Bayes [1, 7, 28], and recently to deep learning based approaches [23, 34]. Premise selection is a classification problem, whereas our proposition generation task is a generation problem with a countably infinite search space.

Sequence generation tasks have been proposed for mathematical reasoning [30, 44]. Our task is different from the previous ones in the sense that ours is non-synthetic, with realistic vocabulary size (i.e., 28K vs. less than 100) and cover various topics in research-level mathematics and computer science that have no general algorithmic solutions.

Synthesising conjectures [24, 31, 54] have been explored before, but mostly not learning-based and limited to specific domains (e.g. in inductive proofs): propositions are enumerated against some grammars or templates, and then checked by automatic theorem provers. Numerous heuristics have been invented to make the enumeration process tractable. Statistical conjecture synthesis based on substitution has been explored in Metamath [47] and earlier in Mizar [12]. In comparison, propositions in our work are synthesised purely from input tokens (without referring to semantic operations like substitution).

Seq2seq models have been used by Wang et al. [48] to map informal LaTeX source to mathematical statements in the Mizar system.

9 Conclusion

We have defined a novel proposition generation task on a large-scale dataset mined from mechanised declarative proofs. We have benchmarked existing seq2seq models on this dataset and their performance shows their impressive reasoning capability despite the difficulty of the task. To further push the limit, we proposed HAT — a novel hierarchical adaptation of the transformer — that outperforms the transformer baseline by a large margin. The dataset is of significance both to the proof assistant community (for promoting automation) and to the machine learning community (for benchmarking the reasoning capability of new models).

10 Broader Impact

In the short term, proof assistants are mainly used to ensure rigorous reasoning and build trust: mathematicians (software engineers) can confidently build their proofs (systems) on top of formally verified ones without worrying about fatal errors. Our research will facilitate the laborious mechanisation process. In the long term, advance in effective reasoning within proof assistants will potentially lead to auto-programming and auto-mathematics. It may be a concern that machines will displace some engineers and mathematicians. However, we believe that such advance will mainly complement humans by allowing them to focus on the creative parts and leave routine reasoning being automated by machine learning algorithms.

References

[1] Jesse Alama, Tom Heskes, Daniel Kühlwein, Evgeni Tsivtsivadze, and Josef Urban. Premise selection for mathematics by corpus analysis and kernel methods. *Journal of Automated Reasoning*, 52(2):191–213, 2014.

[2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *Proceedings of International Conference on Learning Representations (ICLR)*, 2015.

[3] Kshitij Bansal, Sarah M. Loos, Markus N. Rabe, Christian Szegedy, and Stewart Wilcox. HOList: An Environment for Machine Learning of Higher Order Logic Theorem Proving. In *Proceedings of International Conference on Machine Learning (ICML)*, 2019.

[4] Clark Barrett, Christopher L Conway, Morgan Deters, Liana Hadarean, Dejan Jovanović, Tim King, Andrew Reynolds, and Cesare Tinelli. CVC4. In *Proceedings of International Conference on Computer Aided Verification*, 2011.

[5] Yves Bertot and Pierre Castéran. *Interactive theorem proving and program development: Coq'Art: the calculus of inductive constructions*. Springer Science & Business Media, 2013.

[6] Jasmin Christian Blanchette, Sascha Böhme, and Lawrence C Paulson. Extending Sledgehammer with SMT solvers. In *Proceedings of International Conference on Automated Deduction*, 2011.

[7] Jasmin Christian Blanchette, Cezary Kaliszyk, Lawrence C Paulson, and Josef Urban. Hammering towards QED. *Journal of Formalized Reasoning*, 9(1):101–148, 2016.

[8] Łukasz Czajka and Cezary Kaliszyk. Hammer for Coq: Automation for dependent type theory. *Journal of automated reasoning*, 61(1-4):423–453, 2018.

[9] Leonardo De Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Proceedings of International conference on Tools and Algorithms for the Construction and Analysis of Systems*, 2008.

[10] Manuel Eberl. Randomised Binary Search Trees. *Archive of Formal Proofs*, 2018. ISSN 2150-914x. `http://isa-afp.org/entries/Randomised_BSTs.html`, Formal proof development.

[11] Manuel Eberl and Lawrence C. Paulson. The Prime Number Theorem. *Archive of Formal Proofs*, 2018. ISSN 2150-914x. `http://isa-afp.org/entries/Prime_Number_Theorem.html`, Formal proof development.

[12] Thibault Gauthier, Cezary Kaliszyk, and Josef Urban. Initial experiments with statistical conjecturing over large formal corpora. In *Joint Proceedings of the FM4M, MathUI, and ThEdu Workshops, Doctoral Program, and Work in Progress at the Conference on Intelligent Computer Mathematics 2016 (CICM-WiP 2016)*, 2016.

[13] Thibault Gauthier, Cezary Kaliszyk, and Josef Urban. TacticToe: Learning to Reason with HOL4 Tactics. In *Proceedings of International Conference on Logic for Programming, Artificial Intelligence and Reasoning (LPAR)*, 2017.

[14] Victor BF Gomes, Martin Kleppmann, Dominic P Mulligan, and Alastair R Beresford. Verifying strong eventual consistency in distributed systems. *Proceedings of the ACM on Programming Languages*, 2017.

[15] Georges Gonthier and Assia Mahboubi. An introduction to small scale reflection in Coq. *Journal of Formalized Reasoning*, 3(2):95–152, 2010.

[16] Georges Gonthier, Andrea Asperti, Jeremy Avigad, Yves Bertot, Cyril Cohen, François Garillot, Stéphane Le Roux, Assia Mahboubi, Russell O'Connor, Sidi Ould Biha, et al. A machine-checked proof of the odd order theorem. In *Proceedings of International Conference on Interactive Theorem Proving*, 2013.

[17] Michael J. Gordon, Arthur J. Milner, and Christopher P. Wadsworth. *Edinburgh LCF: A Mechanised Logic of Computation*. Lecture Notes in Computer Science. Springer-Verlag Berlin Heidelberg, 1979.

[18] Sébastien Gouëzel. Subadditive cocycles and horofunctions. In *Proceedings of the International Congress of Mathematicians (ICM)*, 2017.

[19] Thomas Hales, Mark Adams, Gertrud Bauer, Tat Dat Dang, John Harrison, Hoang Le Truong, Cezary Kaliszyk, Victor Magron, Sean McLaughlin, Tat Thang Nguyen, et al. A formal proof of the Kepler conjecture. In *Forum of Mathematics, Pi*, volume 5. Cambridge University Press, 2017.

[20] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8): 1735–1780, 1997.

[21] Kryštof Hoder and Andrei Voronkov. Sine Qua Non for Large Theory Reasoning. In *Proceedings of International Conference on Automated Deduction*, 2011.

[22] Daniel Huang, Prafulla Dhariwal, Dawn Song, and Ilya Sutskever. GamePad: A Learning Environment for Theorem Proving. In *Proceedings of International Conference on Learning Representations (ICLR)*, 2019.

[23] Geoffrey Irving, Christian Szegedy, Alexander A. Alemi, Niklas Eén, François Chollet, and Josef Urban. DeepMath - Deep Sequence Models for Premise Selection. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.

[24] Moa Johansson, Lucas Dixon, and Alan Bundy. Conjecture Synthesis for Inductive Theories. *Journal of Automated Reasoning*, 47(3):251–289, 2011.

[25] Cezary Kaliszyk, François Chollet, and Christian Szegedy. HolStep: A Machine Learning Dataset for Higher-order Logic Theorem Proving. In *Proceedings of International Conference on Learning Representations (ICLR)*, 2017.

[26] Emin Karayel and Edgar Gonzàlez. Strong Eventual Consistency of the Collaborative Editing Framework WOOT. *Archive of Formal Proofs*, 2020. ISSN 2150-914x. `http://isa-afp.org/entries/WOOT_Strong_Eventual_Consistency.html`, Formal proof development.

[27] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, et al. seL4: Formal verification of an OS kernel. In *Proceedings of the ACM SIGOPS symposium on Operating systems principles*, 2009.

[28] Daniel Kühlwein, Jasmin Christian Blanchette, Cezary Kaliszyk, and Josef Urban. MaSh: Machine Learning for Sledgehammer. In *Proceedings of International Conference on Interactive Theorem Proving*, 2013.

[29] Peter Lammich and Tobias Nipkow. Priority Search Trees. *Archive of Formal Proofs*, 2019. ISSN 2150-914x. `http://isa-afp.org/entries/Priority_Search_Trees.html`, Formal proof development.

[30] Guillaume Lample and François Charton. Deep learning for symbolic mathematics. In *Proceedings of International Conference on Learning Representations (ICLR)*, 2020.

[31] Craig E Larson. A survey of research in automated mathematical conjecture-making. *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, 69:297, 2005.

[32] Xavier Leroy. Formal Verification of a Realistic Compiler. *Communications of the ACM*, 52(7): 107–115, 2009.

[33] Andreas Lochbihler. CryptHOL. *Archive of Formal Proofs*, 2017. ISSN 2150-914x. `http://isa-afp.org/entries/CryptHOL.html`, Formal proof development.

[34] Sarah M. Loos, Geoffrey Irving, Christian Szegedy, and Cezary Kaliszyk. Deep Network Guided Proof Search. In *International Conference on Logic for Programming, Artificial Intelligence and Reasoning (LPAR)*, 2017.

[35] Jia Meng and Lawrence C Paulson. Lightweight relevance filtering for machine-generated resolution problems. *Journal of Applied Logic*, 7(1):41–57, 2009.

[36] Benedikt Nordhoff, Stefan Körner, and Peter Lammich. Finger Trees. *Archive of Formal Proofs*, 2010. ISSN 2150-914x. `http://isa-afp.org/entries/Finger-Trees.html`, Formal proof development.

[37] Aditya Paliwal, Sarah Loos, Markus Rabe, Kshitij Bansal, and Christian Szegedy. Graph Representations for Higher-Order Logic and Theorem Proving, 2019. URL `http://arxiv.org/abs/1905.10006`.

[38] Lawrence C Paulson. *Isabelle: A generic theorem prover*, volume 828. Springer Science & Business Media, 1994.

[39] Lawrence C. Paulson. Gödel's Incompleteness Theorems. *Archive of Formal Proofs*, 2013. ISSN 2150-914x. `http://isa-afp.org/entries/Incompleteness.html`, Formal proof development.

[40] George Pîrlea and Ilya Sergey. Mechanising blockchain consensus. In *Proceedings of ACM SIGPLAN International Conference on Certified Programs and Proofs*, 2018.

[41] Alexandre Riazanov and Andrei Voronkov. The design and implementation of VAMPIRE. *AI communications*, 15(2, 3):91–110, 2002.

[42] Piotr Rudnicki. An overview of the Mizar project. In *Proceedings of the 1992 Workshop on Types for Proofs and Programs*, pages 311–330. Citeseer, 1992.

[43] Alex Sanchez-Stern, Yousef Alhessi, Lawrence Saul, and Sorin Lerner. Generating Correctness Proofs with Neural Networks, 2019. URL `http://arxiv.org/abs/1907.07794`.

[44] David Saxton, Edward Grefenstette, Felix Hill, and Pushmeet Kohli. Analysing mathematical reasoning abilities of neural models. In *Proceedings of International Conference on Learning Representations (ICLR)*, 2019.

[45] René Thiemann and Akihisa Yamada. Polynomial Factorization. *Archive of Formal Proofs*, 2016. ISSN 2150-914x. http://isa-afp.org/entries/Polynomial_Factorization.html, Formal proof development.

[46] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[47] Mingzhe Wang and Jia Deng. Learning to Prove Theorems by Learning to Generate Theorems, 2020. URL https://arxiv.org/abs/2002.07019.

[48] Qingxiang Wang, Chad Brown, Cezary Kaliszyk, and Josef Urban. Exploration of neural machine translation in autoformalization of mathematics in mizar. *Proceedings of ACM SIGPLAN International Conference on Certified Programs and Proofs*, 2020.

[49] Makarius Wenzel. The Isabelle/Isar Implementation. https://isabelle.in.tum.de/dist/Isabelle2020/doc/isar-ref.pdf, 2020. [Online; accessed 31-May-2020].

[50] Markus M Wenzel. *Isabelle/Isar—a versatile environment for human-readable formal proof documents*. PhD thesis, Technische Universität München, 2002.

[51] Freek Wiedijk. Mizar light for HOL light. In *International Conference on Theorem Proving in Higher Order Logics*, pages 378–393. Springer, 2001.

[52] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, Jeff Klingner, Apurva Shah, Melvin Johnson, Xiaobing Liu, Lukasz Kaiser, Stephan Gouws, Yoshikiyo Kato, Taku Kudo, Hideto Kazawa, Keith Stevens, George Kurian, Nishant Patil, Wei Wang, Cliff Young, Jason Smith, Jason Riesa, Alex Rudnick, Oriol Vinyals, Greg Corrado, Macduff Hughes, and Jeffrey Dean. Google's neural machine translation system: Bridging the gap between human and machine translation. *CoRR*, abs/1609.08144, 2016. URL http://arxiv.org/abs/1609.08144.

[53] Kaiyu Yang and Jia Deng. Learning to Prove Theorems via Interacting with Proof Assistants. In *Proceedings of International Conference on Machine Learning (ICML)*, 2019.

[54] Weikun Yang, Grigory Fedyukovich, and Aarti Gupta. Lemma Synthesis for Automating Induction over Algebraic Data Types. In *Proceedings of International Conference on Principles and Practice of Constraint Programming*, 2019.

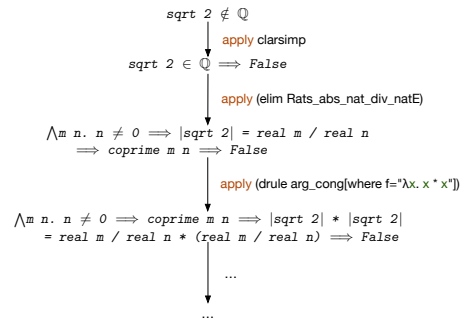

Figure 7: Proof state transition after applying tactics

```
lemma Rats_abs_nat_div_natE:
  assumes "x ∈ ℚ"
  obtains m n :: nat where "n ≠ 0" and "¦x¦ = real m / real n"
    and "coprime m n"
```

Figure 8: A proved lemma in the library of Isabelle/HOL claiming that given a rational number x there exists a coprime pair of natural numbers m and n such that $\mid x \mid = m/n$.

A From Tactic Proofs to Declarative Ones

```
let IRRATIONAL_SQRT_NONSQUARE = prove
 (`!n. rational(sqrt(&n)) ==> ?m. n = m EXP 2`,
  REWRITE_TAC[rational] THEN REPEAT STRIP_TAC THEN
  FIRST_ASSUM(MP_TAC o AP_TERM `\x:real. x pow 2`) THEN
  SIMP_TAC[SQRT_POW_2; REAL_POS] THEN
  ONCE_REWRITE_TAC[GSYM REAL_POW2_ABS] THEN
  REPEAT(FIRST_X_ASSUM(STRIP_ASSUME_TAC o GEN_REWRITE_RULE I [integer])) THEN
  ASM_REWRITE_TAC[REAL_ABS_DIV] THEN DISCH_THEN(MP_TAC o MATCH_MP(REAL_FIELD
   `p = (n / m) pow 2 ==> ~(m = &0) ==> m pow 2 * p = n pow 2`)) THEN
  ANTS_TAC THENL [ASM_MESON_TAC[REAL_ABS_ZERO]; ALL_TAC] THEN
  REWRITE_TAC[REAL_OF_NUM_POW; REAL_OF_NUM_MUL; REAL_OF_NUM_EQ] THEN
  ASM_MESON_TAC[EXP_MULT_EXISTS; REAL_ABS_ZERO; REAL_OF_NUM_EQ]);;

(* ------------------------------------------------------------------------- *)
(* In particular, prime numbers.                                             *)
(* ------------------------------------------------------------------------- *)

let IRRATIONAL_SQRT_PRIME = prove
 (`!p. prime p ==> ~rational(sqrt(&p))`,
  GEN_TAC THEN ONCE_REWRITE_TAC[GSYM CONTRAPOS_THM] THEN REWRITE_TAC[] THEN
  DISCH_THEN(CHOOSE_THEN SUBST1_TAC o MATCH_MP IRRATIONAL_SQRT_NONSQUARE) THEN
  REWRITE_TAC[PRIME_EXP; ARITH_EQ]);;

(* ------------------------------------------------------------------------- *)
(* In particular, sqrt(2) is irrational.                                     *)
(* ------------------------------------------------------------------------- *)

let IRRATIONAL_SQRT_2 = prove
 (`~rational(sqrt(&2))`,
  SIMP_TAC[IRRATIONAL_SQRT_PRIME; PRIME_2]);;
```

```
Theorem new_equality :
 (3 * p - 4 * q) * (3 * p - 4 * q) = 2 * ((3 * q - 2 * p) * (3 * q - 2 * p)).
repeat rewrite sub_square_identity; auto with arith.
repeat rewrite square_recompose; rewrite mult_minus_distr_l.
apply minus_eq_decompose; try rewrite hyp_sqrt; ring.
Qed.
End sqrt2_decrease.
Hint Resolve lt_le_weak comparison2: sqrt.

Theorem sqrt2_not_rational :
 forall p q : nat, q <> 0 -> p * p = 2 * (q * q) -> False.
intros p q; generalize p; clear p; elim q using (well_founded_ind lt_wf).
clear q; intros q Hrec p Hneq; generalize (neq_0_lt _ (sym_not_equal Hneq));
 intros Hlt_0_q Heq.
apply (Hrec (3 * q - 2 * p) (comparison4 _ _ Hlt_0_q Heq) (3 * p - 4 * q)).
apply sym_not_equal; apply lt_neq; apply plus_lt_reg_l with (2 * p);
 rewrite <- plus_n_O; rewrite <- le_plus_minus; auto with *.
apply new_equality; auto.
Qed.
```

Figure 9: Tactic-style proof of the irrationality of $\sqrt{2}$ in HOL Light (left) and Coq (right)

In the early days of proof assistants [17], people were generally concerned whether a theorem can be mechanically checked, which is essentially a process of transforming the *proof state* until all subgoals have been discharged. Tactics, due to their direct access to proof states and their programmable nature (i.e., primitive tactics can be easily composed into sophisticated ones), have become the building blocks of most proof assistants.

To illustrate the interaction between tactics and proof states, we briefly examine the tactic proof of the irrationality of $\sqrt{2}$ in Isabelle/HOL. Given the statement `theorem "sqrt 2 ∉ ℚ"`, the initial proof state is the conjecture itself – `sqrt 2 ∉ ℚ`. We then consider to deploy a proof-by-contradiction strategy and applied the `clarsimp` tactic (see Fig. 7), which transformed the state to `sqrt 2 ∈ ℚ ⟹ False` (i.e., assuming $\sqrt{2}$ is rational, we want to derive false). To utilise the assumption, we find a relevant lemma called `Rats_abs_nat_div_natE` in the library (see Fig. 8), and then update the state by applying the `elim` tactic argumented with this found lemma. The resulted state requires us to derive false with fixed coprime natural numbers m and n such that $n \neq 0$ and

$$\mid \sqrt{2} \mid = \frac{m}{n}. \tag{3}$$

Subsequently, we square both sides of (3) by invoking another tactic `drule` with the argument `arg_cong[where f="λx. x * x"]`, and obtain a new state. We repeat this process until we reach a state with no subgoal left, and the final mechanised proof is a sequence of tactics possibly with arguments as displayed in §2.

The proof of the irrationality of $\sqrt{2}$ is also available in other tactic-based proof assistants including HOL Light[7] and Coq[8] (see Fig. 9). The proof scripts are similar to the tactic-based ones in Isabelle/HOL except that tactic combinators (which combines several tactics to produce a new one) are used more heavily.

In general, a tactic proof can be viewed as a sequence of commands issued to the proof assistant instructing how the proof state should be transformed. In this case, people are mainly interested in the theorem being checked leaving the proof itself like incomprehensible machine code.

Pioneered by the Mizar system [42], it has been realised that a human-oriented declarative proof can become an object of interest itself. The declarative feature (i.e. focusing on intermediate propositions) was then incorporated into other systems including HOL Light [51], Isabelle [50], and Coq [15]. Nowadays mechanised proofs are developed in a mixed manner: tactic-style parts are for finely controlling the proof state (like an assembly language), and declarative parts are for sketching high-level reasoning (like the Python language). Nevertheless, due to technical (e.g. automation) and cultural reasons, proof style varies from one system to another. Proofs in Isabelle and Mizar are arguably much more declarative than that in other systems — this is in contrast to HOL Light, where most of its proofs are of tactic style despite having a declarative mode.

B Examples of correct synthesises

In this section, we present some correctly synthesised propositions which will be labelled as **F.1** in each case.

F.1 :

$$\text{additive}(\mathcal{A}_{x_1}, \mu_{x_0}) \tag{4}$$

F.2 :

$$\text{subalgebra}(x_0, x_1) \tag{5}$$

F.3 :

$$\text{measure_space}(\mathcal{X}_{x_0}, \mathcal{A}_{x_1}, \mu_{x_0}) \tag{6}$$

F.4 :

$$\sigma(\mathcal{X}_{x_0}, \mathcal{A}_{x_1}) \tag{7}$$

$$\text{positive}(\mathcal{A}_{x_1}, \mu_{x_0}) \tag{8}$$

$$\text{measure_space}(v_1, v_0, v_2) = (\sigma(v_1, v_0) \wedge \text{positive}(v_0, v_2) \wedge \text{additive}(v_0, v_2)) \tag{9}$$

Here, x_0 and x_1 are measure spaces. For a measure space y, $\mathcal{X}_y$, $\mathcal{A}_y$, and μ_y are the three components of y (i.e., $y = (\mathcal{X}_y, \mathcal{A}_y, \mu_y)$), where $\mathcal{X}_y$ is the carrier set, $\mathcal{A}_y$ is a collection of subsets on $\mathcal{X}_y$, and μ_y is a measure defined on $\mathcal{A}_y$. Eq. (5) $\Rightarrow$ (4): x_1 being a subalgebra of x_0 (i.e., (5)) implies $\mathcal{A}_{x_1} \subseteq \mathcal{A}_{x_0}$, so that μ_{x_1} in $\text{additive}(\mathcal{A}_{x_1}, \mu_{x_1})$ (i.e., μ_{x_1} is countably addictive on $\mathcal{A}_{x_1}$ which is implied by x_1 being a measure space) can be substituted with μ_{x_0} which yields (4). Eq. (4) $\Rightarrow$ (6): deriving (6) requires unfolding the definition of measure spaces (i.e., (9)), which requires v_0 is a sigma algebra on v_1, the measure v_2 is non-negative on v_0, and v_2 is countably additive on v_0. Two of the three requirements have already been satisfied by (7) and (8) respectively, while (4) entails the last one and eventually leads to (6).

[7] Example available from `https://github.com/jrh13/hol-light/blob/master/100/sqrt.ml`

[8] Example available from `https://coq.inria.fr/distrib/8.2/contribs/QArithSternBrocot.sqrt2.html`

F.1 :

$$\mathrm{Bseq}(\lambda x.\, x_2(x) x_0^x) \tag{10}$$

F.2 :

$$\mathrm{Cauchy}(\lambda x.\, x_2(x) x_0^x) \tag{11}$$

F.3 :

$$\forall x.\ \mid x_2(x) x_0^x \mid \leq x_3 \tag{12}$$

$$x_3 > 0 \tag{13}$$

F.4 :

$$\mathrm{Bseq}(v_0) = (\exists x > 0. \forall y.\ \mid v_0(y) \mid \leq x) \tag{14}$$

Eq. (11) $\Rightarrow$ (10): a Cauchy sequence is naturally a bounded sequence (i.e., $\mathrm{Cauchy}(x) \vdash \mathrm{Bseq}(x)$). Eq. (10) $\Rightarrow$ (12 - 13) by unfolding the definition of bounded sequences (i.e., (14)).

F.1 :

$$x_{29} \notin \mathrm{path_image}(x_7) \tag{15}$$

F.2 :

$$x_{29} \in \mathrm{proots}(x_0) - \mathrm{proots_within}(x_0, \mathrm{box}(x_1, x_2)) \tag{16}$$

$$\mathrm{path_image}(x_7) \cap \mathrm{proots}(x_0) = \{\} \tag{17}$$

F.3 :

$$x_{29} \notin \mathrm{cbox}(x_1, x_2) \tag{18}$$

F.4 :

$$\mathrm{cbox}(x_1, x_2) = \mathrm{box}(\mathrm{x}_1, \mathrm{x}_2) \cup \mathrm{path_image}(x_7) \tag{19}$$

$$x_{29} \in \mathrm{proots}(x_0) - \mathrm{proots_within}(x_0, \mathrm{box}(x_1, x_2)) \tag{20}$$

Here, $\mathrm{path_image}(x_7)$ is the image of the path function x_7 on the interval $[0, 1]$; $\mathrm{proots}(x_0)$ and $\mathrm{proots_within}(x_0, S)$ are, respectively, the roots of a polynomial x_0 and the roots (of x_0) within a set S; $\mathrm{box}(x_1, x_2) = \{x \mid x_1 < x < x_2\}$ and $\mathrm{cbox}(x_1, x_2) = \{x \mid x_1 \leq x \leq x_2\}$ are (bounded) boxes on an Euclidean space. Eq. (16 - 17) $\Rightarrow$ (15): x_{29} is a root of x_0 (by (16)) that does not intersect with the path of x_7 (i.e., (17)). Eq. (15) $\Rightarrow$ (18): combining with (19), (18) is equivalent to $x_{29} \notin \mathrm{box}(\mathrm{x}_1, \mathrm{x}_2) \wedge x_{29} \notin \mathrm{path_image}(x_7)$, which follows from joining (20) with (15).

F.1 :

$$\frac{x_4(2x_{10})}{x_4(x_{10})} \leq x_9 \tag{21}$$

F.2 :

$$x_9 = \mathrm{Max}\left\{\frac{x_4(2y)}{x_4(y)} \mid y \leq x_8\right\} \tag{22}$$

$$x_{10} \leq x_8 \tag{23}$$

F.3 :

$$x_4(2x_{10}) \leq x_9 x_4(x_{10}) \tag{24}$$

F.4 :

$$0 < x_4(x_{10}) \tag{25}$$

Eq. (22 - 23) $\Rightarrow$ (21): (23) implies

$$\frac{x_4(2x_{10})}{x_4(x_{10})} \in \left\{\frac{x_4(2y)}{x_4(y)} \mid y \leq x_8\right\},$$

hence (21) by the definition of Max. Eq. (21) $\Rightarrow$ (24) by arithmetic and the positivity of the denominator (i.e., (25)).

F.1 :

$$\lim_{x\to\infty}(x_4(x) - x_5(x)) = \pi \tag{26}$$

F.2 :

$$\lim_{x\to\infty} x_5(x) = -\frac{\pi}{2} \tag{27}$$

$$\lim_{x\to\infty} x_4(x) = \frac{\pi}{2} \tag{28}$$

F.3 :

$$\lim_{x\to\infty}\left(\frac{x_4(x) - x_5(x)}{2\pi}\right) = \frac{1}{2} \tag{29}$$

F.4 :

Both (27 - 28) $\Rightarrow$ (26) and (26) $\Rightarrow$ (29) can be derived from limit arithmetic.

F.1 :

$$|ix_0 + \sqrt{1 - x_0^2}| = 1 \tag{30}$$

F.2 :

$$|ix_0 + \sqrt{1 - x_0^2}|^2 = 1 \tag{31}$$

F.3 :

$$\Im(\arcsin(x_0)) = 0 \tag{32}$$

F.4 :

$$\Im(\arcsin(v_0)) = -\ln(|iv_0 + \sqrt{1 - v_0^2}|) \tag{33}$$

$$e^{-v_0} = 1/(e^{v_0}) \tag{34}$$

Eq. (31) $\Rightarrow$ (30) by arithmetic. Eq. (30) $\Rightarrow$ (32):

$$\Im(\arcsin(v_0)) = -\ln(|iv_0 + \sqrt{1 - v_0^2}|) = -\ln 1 = 0.$$

F.1 :

$$x_4 = x_{10}[x_{12}] \tag{35}$$

F.2 :

$$\forall x.\, x_3 = x_6[x] \wedge x < \mathrm{len}(x_6) \longrightarrow x_4 = x_{10}[x] \tag{36}$$

$$x_3 = x_6[x_{12}] \tag{37}$$

$$x_{12} < \mathrm{len}(x_6) \tag{38}$$

F.3 :

$$x_4 = x_7[x_{11}] \tag{39}$$

F.4 :

$$x_7 = x_9 \# x_{10} \tag{40}$$

$$x_{12} < \mathrm{len}[x_6] \tag{41}$$

$$x_{11} = x_{12} + 1 \tag{42}$$

Here, $x_{10}[x_{12}]$ refers to the x_{12}th element in the list x_{10}; len is the length function on a list; $x_9 \# x_{10}$ is a list where the element x_9 is concatenated to the front of the list x_{10}. Eq. (36 - 38) $\Rightarrow$ (35) by instantiating the quantified variable x in (36) to x_{12} and combining with (37 - 38). Eq. (35) $\Rightarrow$ (39):

$$x_4 = x_{10}[x_{12}] = (x_9 \# x_{10})(x_{12} + 1) = x_7[x_{11}].$$

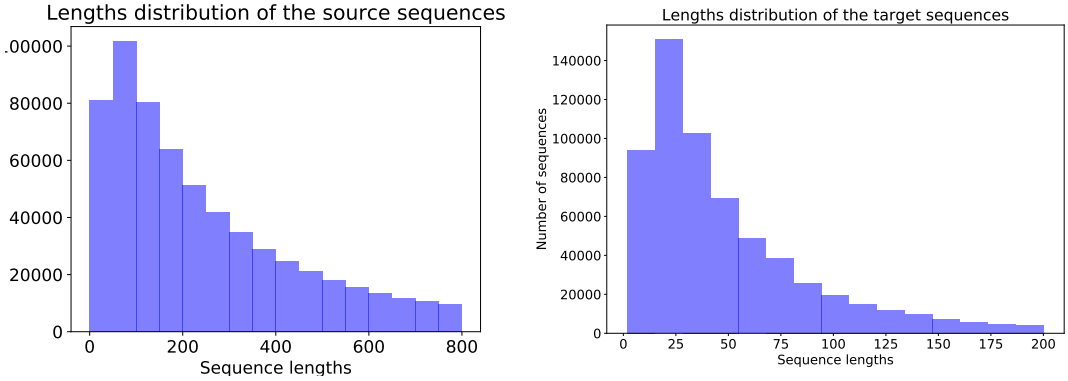

Figure 10: Distribution of source and target sequence lengths for the IsarStep task.

C Distribution of Sequence Lengths

Fig. 10 shows the distribution of the source and target sequence lengths. The source sequence is a concatenation of the source propositions.